

Diya Jain

Dr. Yao Liu

CNT 4419: Secure Coding

27th November, 2023

SQL Injection

Abstract

This lab will dive into SQL injection attacks and their impact on web applications. You'll encounter a vulnerable web app in the lab environment. Your task? Spot the weaknesses, exploit them, and showcase the potential damage from these attacks. Expect to work with SQL statements, SQL injections, and prepared statements. The goal is to equip students with skills to tackle these security threats head-on.

Introduction

Web applications often serve as gateways to databases for storing and retrieving data. Unfortunately, if input isn't properly validated and sanitized, attackers can exploit vulnerabilities by injecting malicious code into SQL queries. This breach, known as SQL injection, can severely compromise data privacy. The aim of this lab is to provide students with an understanding of SQL injection attacks and effective preventive measures to thwart such security risks.

Methods and Materials

To begin this lab, you'll require a designated folder housing the web application, accessible via a specific URL. This folder needs to be accessed through a customized virtual machine, which aligns the URL's domain name with a local IP address. For detailed insights into SQL injection, consulting the SEED Book,

"Computer & Internet Security: A Hands-on Approach, 2nd Edition" by Wenliang Du, will provide valuable information and knowledge.

Results

Task 1: Get Familiar with SQL Statements

The task involves familiarizing ourselves with SQL queries using a database named Users. Within this database, there's a table called 'credential' that holds employee details like eid, name, password, salary, ssn, phone number, address, email, and nickname. Our objective is to create an SQL query that retrieves and displays all information related to an employee named Alice. The SQL query to accomplish this is:

```
SELECT * FROM credential WHERE Name='Alice';
```

This task is straightforward for those accustomed to working with databases and SQL commands.

```
Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql> use sqllab users;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A
```

```
Database changed  
mysql> show tables;  
+-----+  
| Tables_in_sqllab_users |  
+-----+  
| credential              |  
+-----+  
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM credential WHERE Name='Alice';  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
| ID | Name | EID   | Salary | birth | SSN    | PhoneNumber | Address | Email | NickName | Password |  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
| 1  | Alice | 10000 | 20000 | 9/20  | 10211002 |             |         |      |          | fdb6e918bdac83000aa54747fc95fe0470fff4976 |  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

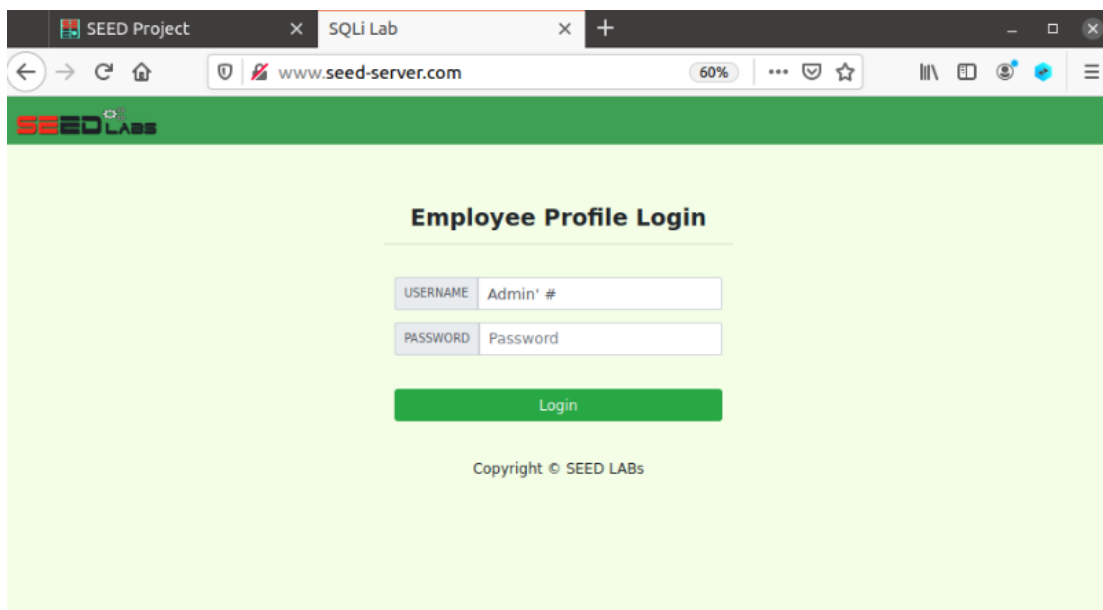
```
mysql>
```

Task 2: SQL Injection Attack on SELECT Statement

Task 2.1: SQL Injection Attack from webpage

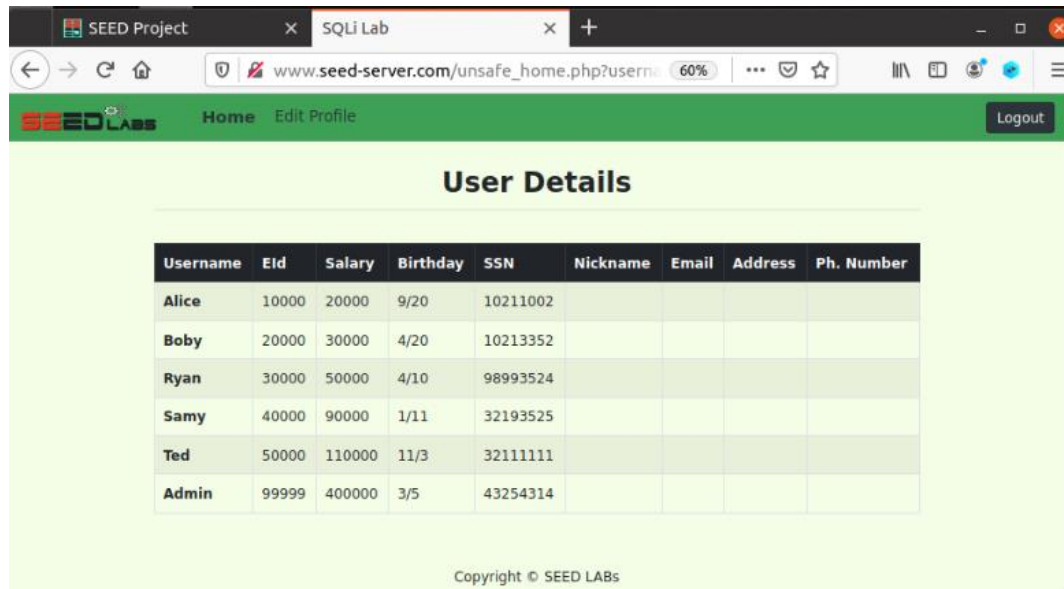
This task delves into understanding SQL injection—a method utilized by attackers to execute harmful SQL commands, gaining access to all information stored within a victim's database. Our starting point is the login page hosted at www.seed-server.com. This page mandates the input of both a username and password for access. In this scenario, we'll adopt the role of an attacker who lacks knowledge about an employee's credentials but aims to breach the login page.

The vulnerability lies within 'unsafe_home.php,' a file responsible for implementing user authentication. While we possess knowledge of the username, the password remains unknown. This situation prompts us to exploit a potential vulnerability and attempt unauthorized access.



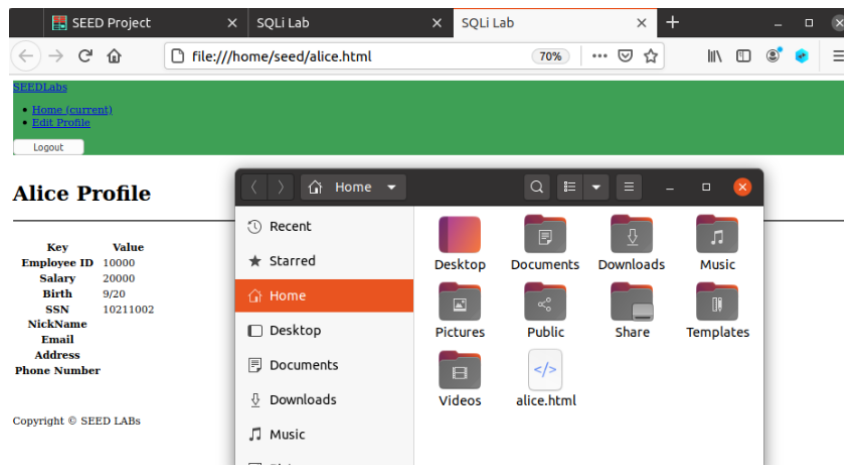
By inputting 'Admin' #' into the USERNAME field, we gain access to view all User Details, mirroring the illustration provided. This method permits logging in as Admin without requiring knowledge of the password, exploiting a SQL

vulnerability that bypasses password verification. Utilizing a combination of a single quotation mark and the pound sign encloses the username, effectively excluding the password's SQL code and enabling unauthorized access.



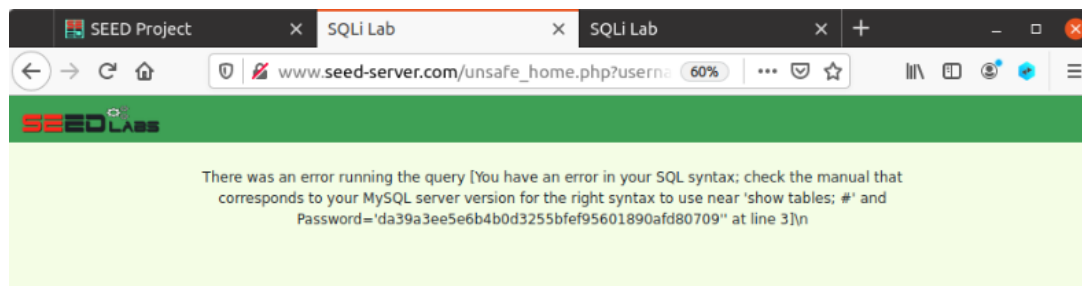
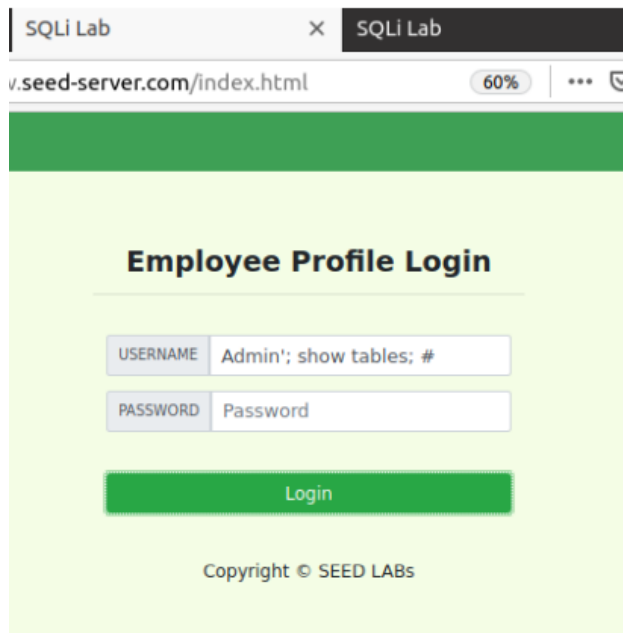
Task 2.2: SQL Injection Attack from command line

In this iteration, we replicate the prior task but bypass the webpage entirely. Instead, we opt for command line tools like 'curl' to dispatch HTTP requests. Subsequently, upon accessing 'alice.html,' we can view Alice's profile.



Task 2.3: Append a new SQL statement

In this lab exercise, we'll explore altering the database utilizing a known vulnerability present in the login page. Employing a SQL injection attack, our aim is to transform a single SQL query into two distinct queries. The webpage's response, as depicted below, displays messages when attempting to employ two SQL queries. This behavior is attributed to the utilization of 'use_query()' within the PHP code.



Enabling the execution of two SQL queries necessitates a shift from using 'query()' to 'multi_query()'. Once this modification is made, we initiate DELETE actions. The visual representation below illustrates the removal of Alice's credentials from

the database. Notably, the password field has been commented out, rendering it inactive within the database.

The screenshot shows the SQLi Lab environment. On the left, a terminal window displays the following MySQL commands and results:

```
Database changed
mysql> show tables;
+-----+
| Tables_in_sqlilab_users |
+-----+
| credential               |
+-----+
1 row in set (0.00 sec)

mysql> select * from credential;
+----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN |
+----+-----+-----+-----+-----+-----+
| 1 | Alice | 10000 | 20000 | 9/20 | 10211092 |
| 2 | Boby | 20000 | 30000 | 4/20 | 10213352 |
| 3 | Ryan | 30000 | 50000 | 4/10 | 98993524 |
| 4 | Samy | 40000 | 90000 | 1/11 | 32193525 |
| 5 | Ted | 50000 | 110000 | 11/3 | 32111111 |
| 6 | Admin | 99999 | 400000 | 3/5 | 43254314 |
+----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)

mysql> mysql> select * from credential;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email | NickName | Password |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | Boby | 20000 | 30000 | 4/20 | 10213352 | | | | | | b78ed97677c161c1c82c142906674ad15242b204 |
| 3 | Ryan | 30000 | 50000 | 4/10 | 98993524 | | | | | | a3c50276cb120637ccab69eb38f19920b017e9ef |
| 4 | Samy | 40000 | 90000 | 1/11 | 32193525 | | | | | | 995b8b8c183f349b3cab0ae7fccc39133508d2af |
| 5 | Ted | 50000 | 110000 | 11/3 | 32111111 | | | | | | 99343bff28a7bb51cb6f22cb20a618701a2c2f58 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

On the right, a web browser window shows the 'Employee Profile Login' page. The 'USERNAME' field contains the injected SQL query: `1st WHERE Name='Alice'; #`. The 'PASSWORD' field is empty. A 'Login' button is visible below the fields.

Task 3: SQL Injection Attack on UPDATE Statement

Task 3.1: Modify your own salary/ Modify your Alice's salary

In this task, we'll delve deeper into the risks associated with SQL injection vulnerabilities, particularly when attackers execute UPDATE statements. This exploit can lead to severe consequences, allowing individuals to modify various details except their own salaries within the database. Assuming the identity of Alice, we aim to alter our salary from 20000 to 50000 by manipulating the query in the NICKNAME box.

To initiate this change, we input our name as the first parameter argument and append our SQL code statement using the '#' symbol. Upon clicking 'save,' the executed query reflects the updated salary information, showcasing the potential for unauthorized modification via SQL injection.

SQLi Lab

www.seed-server.com/unsafe_edit_fi 50%

SEED LABS Home Edit Profile Logout

Alice's Profile Edit

NickName:

Email:

Address:

Phone Number:

Password:

Copyright © SEED LABS

SQLi Lab

www.seed-server.com/unsafe_home.ph 50%

SEED LABS Home Edit Profile Logout

Alice Profile

Key	Value
Employee ID	10000
Salary	50000
Birth	9/20
SSN	10211002
NickName	Alice
Email	
Address	
Phone Number	

Task 3.2: Modify other people's salary

Exploiting SQL injection isn't just limited to altering personal salary information; it also extends to manipulating the salaries of other individuals. Following the same procedure as the previous task, we execute a query to modify our boss, Bobby's, salary to 1. This process demonstrates the capability of SQL injection attacks to impact and modify the salary details of individuals beyond one's own credentials.

Boby's Profile Edit

NickName:

Email:

Address:

Phone Number:

Password:

Copyright © SEED LABS

Boby Profile

Key	Value
Employee ID	20000
Salary	1
Birth	4/20
SSN	10213352
NickName	Boby
Email	
Address	
Phone Number	

Task 3.3: Modify other people's password

Among the riskiest actions in this task is the modification of others' passwords. Our initial step involves logging into Bobby's profile, yielding identical information as depicted in the image above. Following this, we employ an online SHA1 Hash generator to convert a chosen password. For instance, should we desire to change

the password to "hacker," its SHA1 Hash would be derived as "4DCC4173D80A2817206E196A38F0DBF7850188FF."



hacker

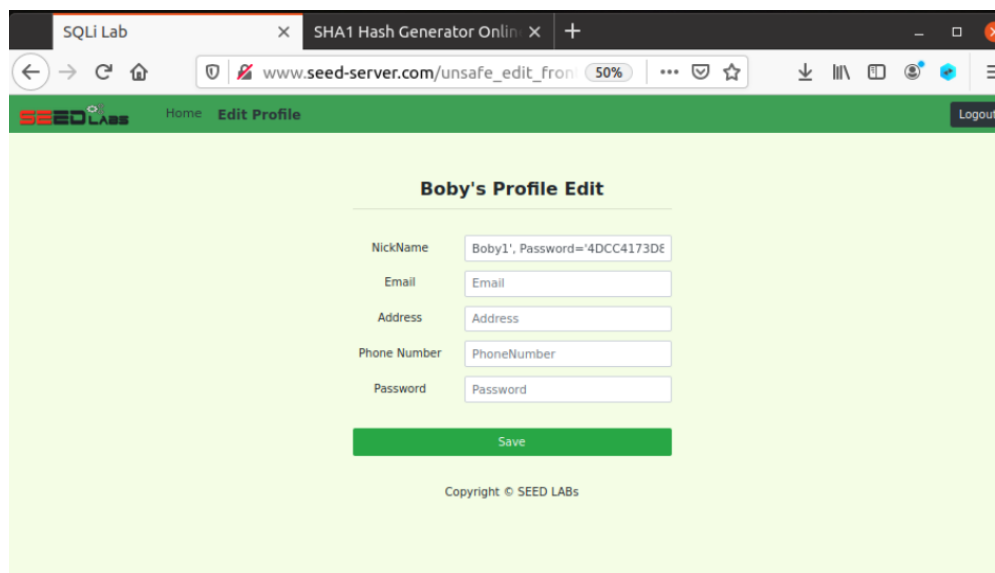
Generate Clear All MD5 SHA256 SHA512 Password Generator

☐ Treat each line as a separate string ☐ Lowercase hash(es)

SHA1 Hash of your string: [Copy to clipboard]

4DCC4173D80A2817206E196A38F0DBF7850188FF

Then, we will do the same steps of the above tasks. First, we will input this query with the new SHA1 Hash password with nickname Bobby1:



SQLi Lab x SHA1 Hash Generator Online x +

www.seed-server.com/unsafe_edit_from 50%

SEED Labs Home Edit Profile Logout

Bobby's Profile Edit

NickName Bobby1', Password='4DCC4173DE

Email Email

Address Address

Phone Number PhoneNumber

Password Password

Save

Copyright © SEED LABS

Observing the changes made, it's evident that the nickname has been altered from Bobby to Bobby1, and the password has been successfully updated. A look into the database confirms that the password for Bobby now aligns with the SHA1 Hash password we implemented, indicating the success of our attack.

Boby Profile	
Key	Value
Employee ID	20000
Salary	1
Birth	4/20
SSN	10213352
NickName	Boby1
Email	
Address	
Phone Number	

```

seed@VM: ~/.../LabsetupSQL
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> use sqlab users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from credential;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email | NickName | Password |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | Alice | 10000 | 20000 | 9/20 | 10211002 | | | | | | fdbe918bdae83000aa54747fc95fe0470fff4976 |
| 2 | Boby | 20000 | 30000 | 4/20 | 10213352 | | | | | | b78ed97677c161c1c82c142906674ad15242b2d4 |
| 3 | Ryan | 30000 | 50000 | 4/10 | 98993524 | | | | | | a3c50276cb120637cca669eb38fb9928b017e9ef |
| 4 | Samy | 40000 | 90000 | 1/11 | 32193525 | | | | | | 995b8b8c183f349b3cab0ae7fccd39133508d2af |
| 5 | Ted | 50000 | 110000 | 11/3 | 32111111 | | | | | | 99343bff28a7bb51cb6f22cb20a618701a2c2f58 |
| 6 | Admin | 99999 | 400000 | 3/5 | 43254314 | | | | | | a5bdf35aldf4ea895905f6f6618e83951a6effc0 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)

mysql> select * from credential where Name='Boby';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email | NickName | Password |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | Boby | 20000 | 1 | 4/20 | 10213352 | | | | | | Boby1 | 4DCC4173D08A2017206E196A38F0DBF7850188FF |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

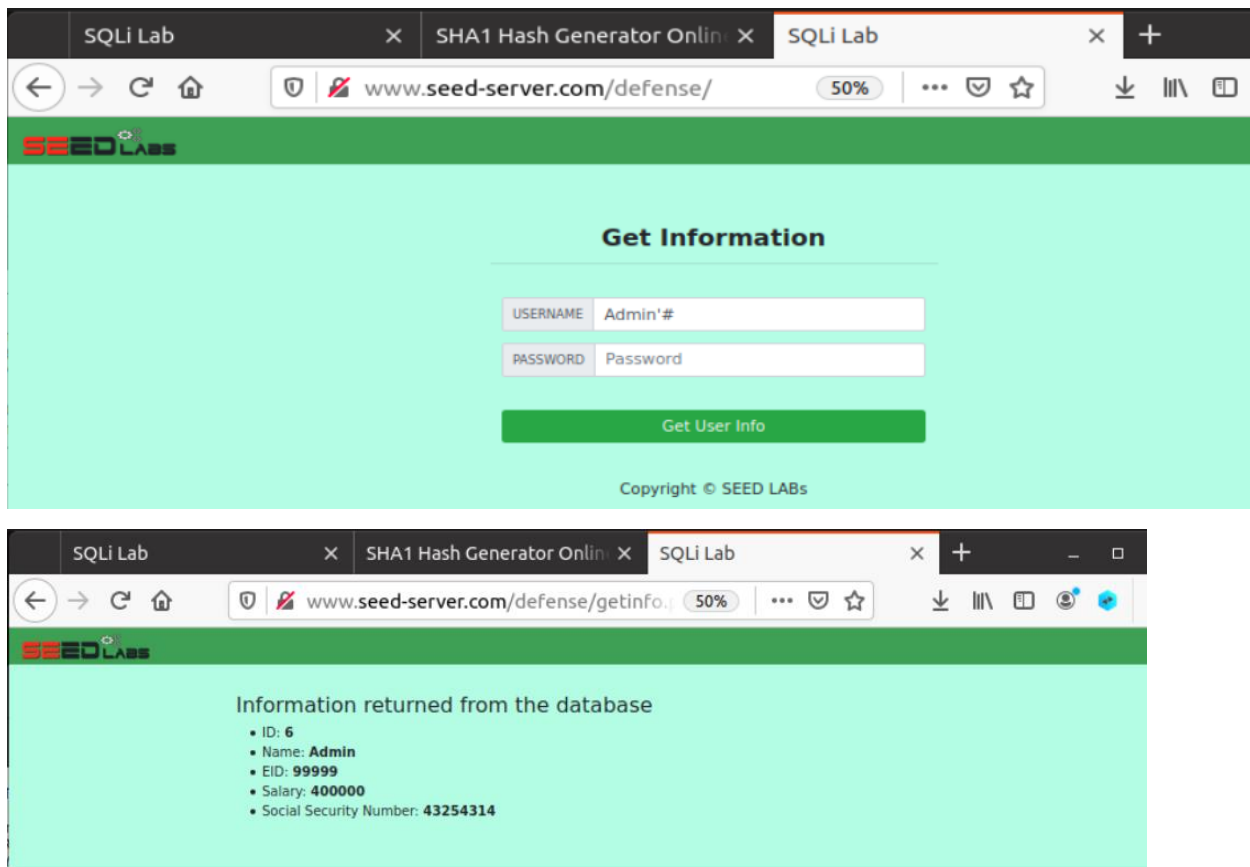
mysql>

```

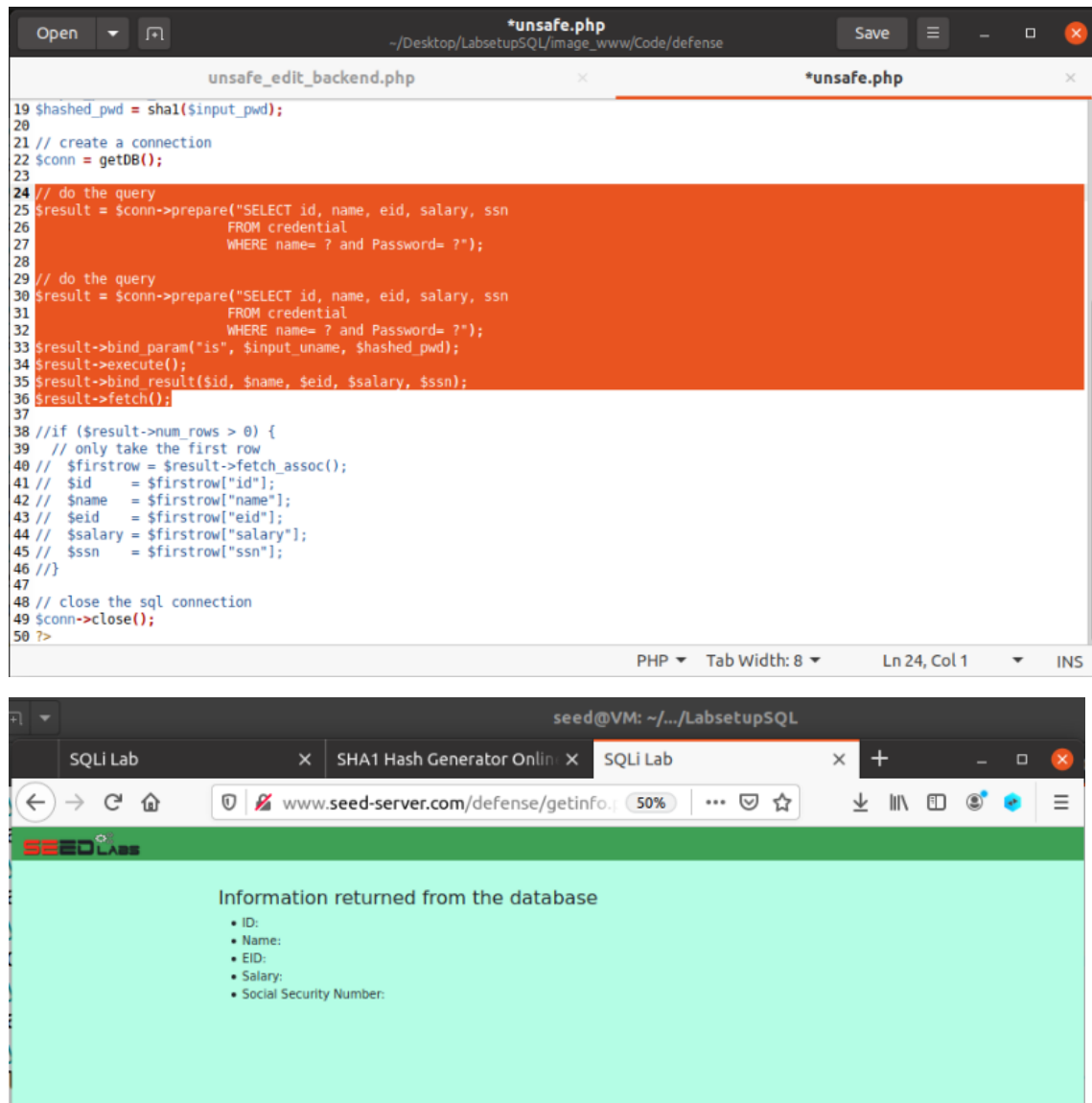
Task 4: Countermeasure – Prepared Statement

Task 4 illustrates the effectiveness of a prepared statement in thwarting SQL injection attacks. Prepared statements serve as predefined templates for SQL queries, featuring placeholders to accommodate user input. These placeholders, typically filled with question marks (?) or named parameters, act as receptacles for user-provided values. When the input values align with these placeholders, the query executes.

To begin, input 'Admin' as the username and proceed by clicking 'Get User Info'. The ensuing display showcases the information retrieved from the database. This step highlights the application of a prepared statement in safeguarding against SQL injection vulnerabilities.



Following that, we'll make modifications to the 'unsafe.php' file in a manner depicted in the image below. Subsequently, after rebuilding and restarting the container, we replicate the previous steps. Upon doing so, the information retrieved from the database appears to be empty. This outcome signifies the success of the alteration made in implementing the countermeasure against the identified vulnerability.



To determine the exploitability of the vulnerability, our initial step involves verifying if we can leverage it effectively. Beginning with a search for an online tool capable of converting the SHA1 Hash stored in the database into the password for the Admin user, we successfully retrieve the password 'seedadmin'.

Sha1 hash calculated hash digest a5bdf35a1df4ea895905f6f6618e83951a6eff c0	Sha1 value Reversed hash value seedadmin
--	---

Subsequently, armed with the acquired password, we input a specific link into the web browser. This action prompts the retrieval of database information, showcasing that despite the prior rectification of SQL injection vulnerabilities, the exploit remains functional.

